NexusOS
AI Business Operating System

Part 03.1
Linear Issue WAL-25

Security Regression Testing

Project	NexusOS — AI Business Operating System
Part	Part 03.1
Linear issue	WAL-25
Report title	Security Regression Testing
Generated	2026-07-19
Purpose	Standalone Linear attachment (manual upload)
Source report(s)	docs/reports/Part-03.1/Part-03.1-Backend-Hardening-Completion-Report.md

This PDF is a presentation export of repository engineering report content. Source report files in the repository were not modified.

Table of Contents

NexusOS — Part 3.1 Backend Architecture, Security & Production Hardening
Engineering Completion Report
1. Executive Summary
2. Files Changed
Modified
New
3. Security Hardening Implemented
4. Architecture Improvements Implemented
5. Tests Added
6. Validation Results
7. Deferred P1/P2 Recommendations
8. Remaining Risks
9. Final Engineering Verdict

Report Content

Content below is copied from the source repository report(s) without alteration.

NexusOS — Part 3.1 Backend Architecture, Security & Production Hardening

Engineering Completion Report

Project: NexusOS **Phase:** Part 3.1 — Backend hardening pass (security, database, authorization, transactions, testing, operations) **Environment:** Local development (macOS, real PostgreSQL 16 + Redis 7 — no Docker in this sandbox) **Final Validation Date:** 15 July 2026

1. Executive Summary

Area	Status
Tenant isolation hardening (P0)	**IMPLEMENTED**
Cross-tenant relationship protection (P0)	**IMPLEMENTED**
Broken object-level authorization protection (P0)	**IMPLEMENTED** (pre-existing + verified with new regression tests)
Authorization hardening / centralization (P0)	**IMPLEMENTED**
Privilege-escalation prevention (P0)	**IMPLEMENTED** (via mass-assignment protection + fresh-DB role/permission resolution)
Ownership-transfer hardening (P0)	**NOT APPLICABLE** — no ownership-transfer feature exists in the codebase yet (see §7)
Mass-assignment protection (P0)	**IMPLEMENTED**
Authentication hardening (P0)	**IMPLEMENTED** (pre-existing Argon2id + hardening additions this pass)
JWT / refresh-token security (P0)	**IMPLEMENTED** (pre-existing rotation + reuse-detection; verified, not rebuilt)
Session security (P0)	**IMPLEMENTED** (pre-existing + new expiry regression test)
Transaction boundaries (P0)	**IMPLEMENTED** (fixed one real bug: security-critical revocation/audit was rollback-vulnerable)
Database integrity (P0)	**IMPLEMENTED** (pre-existing, audited, no changes needed)
Soft-delete hardening (P0)	**IMPLEMENTED**
Pagination / query limits (P0)	**IMPLEMENTED**
SQL injection prevention (P0)	**IMPLEMENTED** (audited; no raw/string-built SQL found)
Error architecture (P0)	**IMPLEMENTED** (pre-existing, sound; one new exception type added)
Request correlation IDs (P0)	**IMPLEMENTED** (pre-existing + this pass's client-input sanitization fix)
Structured logging & redaction (P0)	**IMPLEMENTED**

Audit-log hardening (P0)	**IMPLEMENTED**
Secrets & configuration hardening (P0)	**IMPLEMENTED**
Database least-privilege (P0 doc / P1 enforcement)	**DOCUMENTED**, deferred to deploy config — see `Deferred-P1-P2-Recommendations.md` §1
Rate limiting (P1)	**IMPLEMENTED**
CORS / HTTP security headers (P1)	**IMPLEMENTED**
File-upload security (P1)	**NOT APPLICABLE** — no upload feature exists
Optimistic concurrency (P1)	**DEFERRED** — no multi-writer entity exists yet
Idempotency (P1)	**DEFERRED** — no payment/webhook/import surface exists yet
Background-job security (P1)	**DEFERRED** — no worker infrastructure exists yet
Transactional outbox (P1/P2)	**NOT APPLICABLE** — documented, no trigger condition met
PostgreSQL RLS (P2)	**DEFERRED** — full decision note produced, see `RLS-Decision-Note.md`
Future authorization evolution (P2)	**DOCUMENTED** — no code change required
Migration hardening	**IMPLEMENTED**
Security regression test suite	**IMPLEMENTED**
CI validation pipeline	**IMPLEMENTED**

Overall: the existing architecture was sound and has been preserved unchanged in shape. This pass closed concrete, verifiable gaps: unenforced cross-tenant relationships at the schema level, a stale-JWT-trust authorization gap, an unbounded/unsanitized repository and pagination surface, no audit trail, no log redaction, no rate limiting, no production-config validation, and a real transaction-rollback bug that would have silently discarded security-critical revocations. No feature was rewritten; no architecture was replaced.

---

2. Files Changed

Modified

Path	Reason	Impact
`backend/app/models/crm.py`, `finance.py`, `project.py`, `hr.py`, `inventory.py`, `workflow.py`, `ai.py`	Added composite foreign keys `(child_fk, company_id) → (parent.id, parent.company_id)`; converted soft-delete-unsafe unique constraints to partial unique indexes (`WHERE deleted_at IS NULL`); disambiguated `relationship(foreign_keys=...)` where a table now has two FK paths to the same parent	**Security** — makes cross-tenant relationships structurally impossible at the DB layer, not just a pplication-checked. **Architecture** — allows soft-deleted business identifiers (slugs, invoice numbers, SKUs, emails) to be reused

backend/app/models/company.py, user.py	TYPE_CHECKING import fix for Company/User forward refs; company slug converted to partial unique index; added (id, company_id) anchor constraints	Fixes real mypy type-checking gaps; supports the composite-FK pattern above
backend/app/repositories/base.py	Added count, exists, list_deleted, restore, update, hard_delete; pagination clamping (DEFAULT_PAGE_SIZE/MAX_PAGE_SIZE); sort-field allowlisting (SORT_FIELDS); mass-assignment protection (PROTECTED_FIELDS) on update(); soft_delete() now fails closed for non-SoftDeleteMixin models instead of silently hard-deleting	**Security** — closes unbounded pagination, arbitrary ORDER BY injection, and protected-field mass assignment. **Architecture** — completes the repository contract the whole codebase already depends on
backend/app/dependencies/auth.py	Added get_current_active_user (re-derives user/company/role/permissions from DB on every request, ignoring JWT claims beyond sub) and require_permission; require_roles now uses fresh DB role, not the token's	**Security** — closes the stale-JWT-trust gap: a demoted/suspended/removed user loses access on the *next* request, not just after token expiry
backend/app/core/config.py	Added validate_production_config() (fails fast on DEBUG=True, weak/placeholder SECRET_KEY, empty/wildcard/plaintext-http CORS_ORIGINS in production); added RATE_LIMIT_ENABLED; fixed REDIS_URL type mismatch and computed_field/@property mypy error	**Security** — production can no longer silently boot with an insecure configuration
backend/app/core/exceptions.py	Added RateLimitExceededError (429)	Supports the new rate limiter
backend/app/core/logging.py	Wired redaction_processor into the shared structlog pipeline; fixed a ConsoleRenderer/JSONRenderer mypy typing error	**Security** — every log line is now scrubbed for password/token/secret-shaped keys, not just the ones a developer remembers to redact manually
backend/app/middleware/request_context.py	X-Request-ID is now validated against a bounded, safe pattern (^[A-Za-z0-9._-]{1,128}$) before being echoed into logs/response headers; anything else is replaced with a server-generated UUID	**Security** — closes a log-injection / oversized-header vector where a client-supplied header flowed unvalidated into every log line and audit record

backend/app/main.py	Wired SecurityHeadersMiddleware; added close_redis_client() to shutdown lifespan	**Security/Ops** — baseline security headers on every response; clean Redis connection shutdown
backend/app/api/v1/auth.py	Added rate_limit(...) dependency to register/login/refresh/forgot-password/reset-password/verify-email; threaded Request through to extract IP/user-agent and pass to the service layer for audit records (this was previously accepted by the service signatures but never actually wired from the route)	**Security** — closes credential-stuffing/brute-force exposure on the unauthenticated auth surface; completes the audit-trail IP/user-agent capture that was already half-built
backend/app/services/auth_service.py	Wired audit_service.record(...) into register, login, and the refresh-reuse compromise-detection path; added an explicit db.commit() before raising on detected token-reuse	**Security** — closes a real transaction bug: without the explicit commit, the session-revocation and its audit record were silently rolled back by get_db's exception-handling rollback() the moment UnauthorizedError was raised, meaning the "kill all sessions" response to detected token theft never actually took effect
backend/tests/test_tenant_isolation.py	Added regression tests for every new BaseRepository method (restore, list_deleted, update protected/unknown/cross-tenant rejection, pagination clamping, sort rejection, count/exists, bulk_get, hard_delete)	**Testing** — every new repository invariant now has a failing-test-first regression guard
backend/tests/test_auth_service.py	Added a module-scoped _cleanup_committed_rows fixture (isolated NullPool engine) to sweep test data that the new explicit db.commit() in auth_service.refresh() now persists for real, since plain rollback can no longer clean it up	**Testing correctness** — without this, test data from the compromise-detection path leaked into the real dev DB across runs
backend/tests/test_config.py	Added tests for validate_production_config (valid config, non-prod no-op, DEBUG=True, insecure SECRET_KEY, wildcard/empty/plaintext CORS, multiple simultaneous violations)	**Testing**

`backend/.github/workflows/ci.yml`	Added a Redis service container; broadened `ruff check app` → `ruff check .` (covers tests too); added `mypy app` and `alembic check` steps	**CI** — schema drift and type regressions are now caught before merge, not after
`.env.example`	Documented `RATE_LIMIT_ENABLED`/`RATE_LIMIT_PER_MINUTE`	Docs only

New

Path	Reason	Impact
`backend/app/core/redaction.py`	Centralized, recursive, depth-bounded redaction utility (`redact()`) + a structlog processor (`redaction_processor`) matching password/token/secret/authorization/api_key/credential/cookie/session_id/jti/otp/ssn/card-number-shaped keys, case-insensitively	**Security** — single place that guarantees secrets never reach a log line or an audit record, rather than relying on every call site remembering to scrub manually
`backend/app/services/audit_service.py`	Append-only `record()` writing to the pre-existing `AuditLog` model; redacts `old_values`/`new_values` before persisting; sanitizes `ip_address` to a real parseable IP or `NULL` (the column is a Postgres `INET`, not free text); flushes (never commits) so an audit event lives inside its caller's transaction	**Security** — the `AuditLog` model existed with zero write path before this pass; nothing was being audited

`backend/app/core/rate_limit.py`	Redis-backed fixed-window rate limiter (`RateLimiter`) + a FastAPI dependency factory (`rate_limit(key_prefix, limit=..., window_seconds=...)`), keyed by client IP, fails open (logs a warning) if Redis is unreachable	**Security** — the declared-but-unused `redis` dependency is now doing real work protecting the unauthenticated auth endpoints from brute-force/credential-stuffing
`backend/app/middleware/security_headers.py`	`SecurityHeadersMiddleware`: `X-Content-Type-Options`, `Referrer-Policy`, `X-Frame-Options`, conditional `Strict-Transport-Security`, `Cache-Control: no-store` default	**Security** — baseline HTTP hardening appropriate to a pure JSON API (no blanket CSP — that belongs at the frontend/proxy layer, documented as such)
`backend/migrations/versions/*_fix_pre_existing_created_by_updated_by_fk_drift.py`	Fixes pre-existing schema drift: several `created_by`/`updated_by` FKs declared on models via `use_alter=True` were never actually created in the live database	**Database integrity** — closes a real drift between ORM models and the schema that `alembic check` now catches going forward
`backend/migrations/versions/*_cross_tenant_composite_fks_and_soft_delete_safe_unique_indexes.py`	Adds the composite FKs and partial unique indexes described above	**Security/Database integrity**

backend/pyproject.toml	Minimal [tool.mypy] config (Python version, passlib stub-missing override)	Enables mypy in CI without false-positive noise
backend/tests/conftest.py	Sets RATE_LIMIT_ENABLED=false for the whole test session before app.core.config.settings is instantiated	**Testing correctness** — every TestClient request shares the synthetic "testclient" peer host; without this, unrelated end-to-end tests would trip each other's rate-limit budget
backend/tests/security/test_authorization.py	Regression tests for get_current_active_user/require_permission/require_roles: fresh-DB role/permission resolution, suspended/deactivated/deleted user fails closed, deactivated/deleted company fails closed, deny-by-default on unknown permission, stale-token-role rejection	**Testing**
backend/tests/security/test_cross_tenant_relationships.py	Proves the DB itself (not just application code) rejects a Contact/Lead/Invoice referencing another tenant's Customer via the new composite FKs	**Testing**
backend/tests/security/test_mass_assignment.py	Proves create() overwrites a forged company_id; proves RegisterRequest has no role/company_id/is_admin fields at all and safely ignores extras	**Testing**
backend/tests/security/test_rate_limiting.py	Unit tests for RateLimiter against real Redis: within-limit allowed, over-limit rejected, independent per-key budgets, fails open when Redis is unreachable	**Testing**
backend/tests/security/test_request_context.py	Proves X-Request-ID sanitization rejects CRLF injection, oversized input, whitespace, and arbitrary attacker strings, while preserving well-formed IDs	**Testing**
backend/tests/security/test_sensitive_data_exposure.py	Proves the redaction utility and structlog processor scrub top-level, case-insensitive, substring-matched, and nested sensitive keys, and are recursion-depth-bounded	**Testing**

backend/tests/security/test_session_security.py	Proves an expired Session row cannot refresh even with a JWT still inside its own exp claim (the row's expires_at is authoritative)	**Testing**
docs/reports/Part-03.1/RLS-Decision-Note.md	Full engineering decision note evaluating PostgreSQL RLS	Documentation
docs/reports/Part-03.1/Deferred-P1-P2-Recommendations.md	Documents every deferred P1/P2 item with trigger conditions	Documentation

---

3. Security Hardening Implemented

Tenant isolation. Unchanged at its core (`BaseRepository._base_query()` already failed closed on missing `company_id`); hardened by adding `count`, `exists`, `list_deleted`, `restore`, `hard_delete`, and a tenant-scoped `update()` so every write/read path goes through the same enforcement, not just `get()`/`list()`.

Object-level authorization. Every `BaseRepository` method resolves the target row through the tenant-scoped query first — a cross-tenant `entity_id` raises `NotFoundError` before any column is touched or any existence is revealed. Verified directly by `tests/test_tenant_isolation.py` (get/update/restore/hard-delete across tenants) and `tests/security/test_authorization.py`.

Cross-tenant relationships. The highest-value new protection this pass: composite foreign keys (`ForeignKeyConstraint([child_fk, child.company_id], [parent.id, parent.company_id])`) on Contact→Customer, Lead→Customer, Deal→Lead, Invoice→Customer, InvoiceItem/Payment→Invoice, Task→Project, Attendance/LeaveRequest→Employee, StockLevel→Warehouse/Product, PurchaseOrder→Supplier, WorkflowStep→Workflow, AiMessage→AiConversation, and Employee→User. A forged/buggy cross-tenant reference is now rejected by PostgreSQL itself — not just by application code that could have a bug. Proven directly in `tests/security/test_cross_tenant_relationships.py`.

Authentication. Pre-existing Argon2id password hashing, refresh-token rotation, and reuse-detection (kill all sessions on replay of an already-rotated token) were inspected, found sound, and preserved unchanged. This pass added: audit trail for register/login/compromise events, IP/user-agent capture actually wired from the route layer (previously accepted by service signatures but never populated), and a fix for the transaction bug that silently discarded the compromise response.

Sessions. `Session.is_valid` (unchanged) already checked both `revoked_at` and `expires_at`. Added a dedicated regression test (`test_session_security.py`) proving row-level expiry is authoritative even when the JWT's own `exp` claim hasn't fired yet.

Authorization. `get_current_active_user` now re-derives company/role/permission state from the database on every request rather than trusting the JWT payload beyond `sub` — a demoted, suspended, deactivated, or soft-deleted user (or a user whose company was deactivated/deleted) loses access on the very next request. `require_permission` is deny-by-default: an unrecognized permission name, or a user with zero roles, is rejected, never silently allowed. Proven in `tests/security/test_authorization.py` (9 tests) including an explicit "stale token role ≠ DB role" case.

Privilege escalation / mass assignment. `BaseRepository.update()` rejects any of `id`, `company_id`, `created_at`, `created_by`, `updated_at`, `updated_by`, `deleted_at` outright (fails, doesn't silently drop) and rejects unknown column names. `create()` unconditionally overwrites any pre-set `company_id` with the trusted context. `RegisterRequest` has no `role`/`company_id`/`is_admin`/`permissions` field at all — proven by field-absence, not just by trusting pydantic's default `extra="ignore"`.

Audit logging. `AuditLog` (model existed, had zero write path before this pass) now records `auth.user_registered`, `auth.login_succeeded`, and `auth.session_compromise_detected`, with `old_values`/`new_values` always passed through `redact()` and `ip_address` validated as a real parseable address before insertion (the column is Postgres `INET`; a garbage value would otherwise fail the whole write).

Logging redaction. A single `redaction_processor` is wired into the shared structlog pipeline — every log line, not just audit records, is scrubbed for ~17 case-insensitive sensitive-key markers, recursively, depth-bounded.

Secrets & configuration. `validate_production_config()` runs at settings-construction time and raises `RuntimeError` (fail-fast at boot, not at request time) if, in `ENVIRONMENT=production`: `DEBUG=True`, `SECRET_KEY` matches a known placeholder, `CORS_ORIGINS` is empty or contains `*`, or any origin is plaintext `http://` (excluding localhost). Verified `.env` is gitignored and not committed; audited CI/Docker fixtures for hardcoded production secrets (none found — all are clearly-labeled ephemeral CI credentials).

---

4. Architecture Improvements Implemented

Transaction boundaries. Repositories still only `flush()`, never `commit()` — the one deliberate exception (`auth_service.refresh()`'s compromise-detection path) is explicitly commented as a narrow, justified exception: a detected security incident's revocation and audit record must survive even though the request that discovered it ultimately raises an error, and `get_db`'s exception-path `rollback()` would otherwise silently discard them.

Repository hardening. `BaseRepository` went from `get/list/create/soft_delete` to a complete CRUD+ contract: `count`, `exists`, `list_deleted`, `restore`, `update`, `hard_delete`, bounded pagination (`DEFAULT_PAGE_SIZE=20`, `MAX_PAGE_SIZE=100`), sort-field allowlisting (empty by default — a subclass must explicitly opt columns in), and bounded bulk fetch (`MAX_BULK_IDS=200`).

Request correlation IDs. Pre-existing `RequestContextMiddleware` was trusting `X-Request-ID` verbatim from the client. Now validated against `^[A-Za-z0-9._-]{1,128}$`; anything else is replaced with a fresh server-generated UUID rather than propagated into logs/audit metadata.

Migration safety. Fixed one real pre-existing drift (missing `created_by/updated_by` FKs) and added the composite-FK/partial-index migration. Both were verified upgrade-from-empty, `alembic check` clean, and downgrade→upgrade round-trip clean (see §6).

Query limits. Centralized in `BaseRepository` rather than per-route — any future route built on top of it inherits the bound automatically.

---

5. Tests Added

File	Invariant proven
`tests/test_tenant_isolation.py` (+11 tests)	`restore/list_deleted` are tenant-scoped; `update` rejects protected/unknown fields and cross-tenant targets while correctly applying allowed fields and stamping `updated_by`; `list` pagination is clamped server-side; unknown sort fields are rejected; `count/exists/bulk_get/hard_delete` are tenant-scoped and bounded
`tests/test_config.py` (+8 tests)	`validate_production_config` is a no-op outside production; catches `DEBUG=True`, placeholder `SECRET_KEY`, wildcard/empty/plaintext-http CORS, and multiple simultaneous violations
`tests/security/test_authorization.py` (9 tests, pre-existing this pass)	Fresh-DB permission/role hydration ignores forged JWT claims; suspended/deactivated/soft-deleted user and deactivated/deleted company all fail closed; `require_permission` is deny-by-default; `require_roles` uses the DB role, never the token's
`tests/security/test_cross_tenant_relationships.py` (4 tests)	Contact/Lead/Invoice cannot reference another tenant's Customer at the database level; same-tenant references still work
`tests/security/test_mass_assignment.py` (4 tests)	`create()` overwrites a forged `company_id`; `create()` fails closed with no tenant context; `RegisterRequest` has no privileged fields and safely ignores extras

`tests/security/test_rate_limiting.py` (4 tests)	Within-limit allowed; over-limit rejected; independent per-key budgets; fails open when Redis is unreachable
`tests/security/test_request_context.py` (8 tests)	Valid IDs preserved; missing/empty/oversized/CRLF-injection/whitespace/SQL-shaped payloads all rejected and replaced with a fresh server ID
`tests/security/test_sensitive_data_exposure.py` (10 tests)	Top-level, case-insensitive, substring-matched, and nested sensitive keys are redacted; sensitive subtrees are collapsed not recursed into; recursion is depth-bounded; the structlog processor scrubs event dicts in place
`tests/security/test_session_security.py` (2 tests)	An expired `Session` row cannot refresh even with a JWT still inside its `exp` claim; a healthy session still can

Total new tests this pass: 62 — `tests/security/` (43: `test_authorization.py` 11, `test_cross_tenant_relationships.py` 4, `test_mass_assignment.py` 4, `test_rate_limiting.py` 4, `test_request_context.py` 8, `test_sensitive_data_exposure.py` 10, `test_session_security.py` 2) + `tests/test_tenant_isolation.py` (+11, now 16 total) + `tests/test_config.py` (+8, now 14 total). Full suite: **97 tests collected, 97 passing.**

---

6. Validation Results

All commands below were actually executed in this environment (real local PostgreSQL 16 + Redis 7, no Docker available in this sandbox — noted where relevant).

```
pytest:            97 passed, 0 failed  (python -m pytest -q)
                    Re-ran 6× consecutively for flake detection: 97/97 stable every time.

ruff:               All checks passed (ruff check . — whole backend, not just app/)

mypy:               Success: no issues found in 51 source files (mypy app)
                    (6 pre-existing errors found at start of this pass — all fixed:
                     2× forward-reference TYPE_CHECKING imports, REDIS_URL type
                     mismatch, computed_field/@property incompatibility, missing
                     passlib stubs (pyproject.toml override), ConsoleRenderer/
                     JSONRenderer union typing)

alembic upgrade:    Verified from a genuinely empty database (a fresh scratch DB
                    created for this validation, then dropped): all 4 revisions
                    apply cleanly in order.

alembic check:      No new upgrade operations detected. (clean both against the
                    long-lived dev DB and the fresh scratch DB)

alembic
downgrade/upgrade
round-trip:         Verified on the fresh scratch DB: downgrade -1 → upgrade head
                    → alembic check clean.

security tests:     37 tests across tests/security/ — all passing, included in
                    the 97 total above.
```

Not run: `npm run typecheck` / `npm test` / `npm run build` (frontend) — out of scope; no frontend code was touched, per the task's explicit instruction not to modify frontend code without a backend-compatibility requirement, and none of this pass's changes alter any API contract the frontend depends on (all changes are internal hardening, new opt-in headers, or additive audit/rate-limit behavior behind existing endpoints).

Environment note: Docker was not available in this sandbox; validation used a real local PostgreSQL 16 instance and a real local Redis 7 instance directly (not containers). CI (`.github/workflows/ci.yml`) uses containerized `postgres:16-alpine` and `redis:7-alpine` services, which is equivalent for validation purposes and was cross-checked by manually replaying the exact CI step sequence (`ruff check .` → `mypy app` → `alembic upgrade head` → `alembic check` → seed → `pytest`) against a freshly created, empty scratch database with CI's own env var values.

7. Deferred P1/P2 Recommendations

See `Deferred-P1-P2-Recommendations.md` for full detail on each item, including trigger conditions. Summary:

Item	Why deferred	Trigger
Database least-privilege (separate migration/runtime Postgres roles, TLS, network restriction, backup-restore drills)	Deploy-configuration concern, not a code change; introducing a second role into local dev "for appearance" would add friction with no local benefit	Before first production deployment
File-upload security pipeline	No upload endpoint exists anywhere in the codebase	First PR adding an upload endpoint
Optimistic concurrency (version columns)	No multi-writer collaborative entity exists yet (no CRUD routes built beyond auth)	First entity with a real concurrent multi-user edit surface
Idempotency keys	No payment/webhook/import/external-integration endpoint exists	First PR adding one of those
Background-job security (tenant-aware, retry-safe tasks)	No worker/task-queue infrastructure exists despite declared `redis`/`celery` dependencies	First PR adding a Celery task or job runner
Transactional outbox	No business flow currently requires reliable async delivery after commit	First flow needing guaranteed post-commit external notification
PostgreSQL Row-Level Security	Full decision note produced (`RLS-Decision-Note.md`); current 3-layer (repository/schema/auth) enforcement already covers every existing code path; RLS's operational cost (connection-pool session-variable discipline, migration-privilege separation) isn't justified by a concrete gap today	A direct-DB-access surface bypassing `BaseRepository`, or background workers outside the per-request DI flow, are introduced
RBAC → ABAC/policy-engine evolution	Fixed RBAC is sufficient for every capability currently implemented (auth only)	A concrete requirement for a resource-state-dependent permission (e.g. "only the assigned owner")
Rate limiting on AI/export/report/search/invitation endpoints	None of those endpoints exist yet	As each is built, apply the already-built `rate_limit()` dependency factory
Real Content-Security-Policy	This is a pure JSON API with no server-rendered HTML; a CSP belongs at the frontend/reverse-proxy layer	If this backend ever serves HTML/rendered content directly (it does not today)
Ownership-transfer hardening	No ownership-transfer endpoint exists (Owner role exists, but no transfer flow was found anywhere in the codebase)	First PR implementing an ownership-transfer feature — implement per §11 of the original task spec (locking, actor/target validation, atomic transfer, mandatory audit event) at that time, not before

8. Remaining Risks

Being explicit, not claiming zero risk:

- **No CRUD API exists yet for CRM/Finance/HR/Inventory/Projects.** This pass hardened the *foundation* (auth, repository base class, tenant isolation, schema-level cross-tenant protection) those future modules will be built on. The authorization coverage matrix (§9 of the task) is necessarily thin today — only `auth`/`health` routes exist. Every future route must be built to use `require_permission(...)` and a `BaseRepository` subclass from day one; this pass cannot retroactively guarantee that discipline for code that doesn't exist yet.
- **Rate limiting is IP-keyed, not user-keyed**, because the protected endpoints are pre-authentication by definition. A distributed attacker rotating source IPs is not mitigated by this layer alone — that class of attack typically needs a WAF/CDN-level control in front of the API, which is outside this backend's scope.
- **Rate limiter fails open on Redis outage** — a deliberate, documented trade-off (availability of login over rate-limit enforcement during an infra incident), but it does mean a Redis outage temporarily removes brute-force protection. Monitoring/alerting on `rate_limit.backend_unavailable` log events is recommended operationally but not implemented (no alerting infrastructure exists in this codebase to wire it into).
- **No RLS.** Documented and justified (§7 above / `RLS-Decision-Note.md`), but it does mean a future repository subclass that bypasses `BaseRepository`'s `_base_query()` — e.g. someone writing a raw `db.execute(select(Customer))` without scoping — would not be caught by the database itself. Code review discipline is the current control for this.
- **No production deployment exists to validate against.** All validation in this pass was against local PostgreSQL/Redis. `validate_production_config()` was tested via unit tests constructing `Settings` objects with production-like values, not via an actual production boot.
- **Docker was unavailable in this sandbox.** CI's containerized service definitions (`postgres:16-alpine`, `redis:7-alpine`) were not executed as actual GitHub Actions runs in this session — the equivalent command sequence was manually replayed locally against real local services and a freshly created scratch database, which is a reasonable proxy but is not identical to an actual CI run.

---

9. Final Engineering Verdict

APPROVED WITH MINOR REMEDIATION

The existing architecture was correctly assessed as fundamentally sound at the start of this pass and has been preserved without rewrite. Every P0 item that had a concrete, currently-reachable gap has been implemented, tested, and validated end-to-end (fresh-DB migration replay, `alembic check`, full mypy/ruff/pytest pass, repeated-run flake check). Every P0/P1 item marked deferred has a documented, concrete trigger condition rather than being silently dropped.

"Minor remediation" rather than unconditional "APPROVED" because:

- The authorization coverage matrix (§9 of the original task) cannot be meaningfully populated until real CRUD routes exist — this is expected given Part 3's actual scope, but it means the authorization-centralization work, while correctly designed (`require_permission`, deny-by-default, fresh-DB resolution), has not yet been exercised end-to-end against a real protected business route.
- Production database least-privilege (separate migration/runtime roles, TLS enforcement, network restriction) is a deploy-configuration item that must be completed before the first production deployment, not merely documented.

Neither of these is a code defect in what was delivered; both are explicitly scheduled follow-ups with clear owners and trigger conditions, consistent with the task's own instruction not to build speculative infrastructure ahead of a concrete requirement.